Práctico: Pilas y Colas con Listas Enlazadas usando OpenCode

Consigna general

Para cada ejercicio deberán implementar la solución en **Java** utilizando **OpenCode**.

Antes de pedirle a OpenCode que genere el código, deberán escribir un prompt donde expliquen claramente:

- qué estructura necesitan usar;
- por qué corresponde usar pila o cola;
- cómo debe funcionar internamente;
- qué operaciones debe implementar;
- qué casos especiales debe contemplar;
- qué complejidad esperan para las operaciones principales.

En el código final deberán dejar como comentario el prompt inicial utilizado y aclarar si realizaron ajustes posteriores.

Ejercicio 1: Pila enlazada de enteros

Implementar una pila utilizando lista enlazada simple.

La pila debe permitir:

- apilar elementos;
- desapilar elementos;
- consultar el tope;
- verificar si está vacía;
- buscar si contiene un valor;
- imprimir su contenido.

El prompt debe explicar por qué el `head` representa el tope de la pila y por qué `push` y `pop` deben resolverse en O(1).

Ejercicio 2: Cola enlazada de enteros

Implementar una cola utilizando lista enlazada simple.

La cola debe permitir:

- encolar elementos;
- desencolar elementos;
- consultar el frente;
- verificar si está vacía;
- buscar si contiene un valor;
- imprimir su contenido.

El prompt debe explicar por qué la cola necesita `head` y `tail`, y qué problema aparecería si solamente se usara `head`.

Ejercicio 3: Pila genérica

Modificar la implementación de pila para que pueda almacenar cualquier tipo de dato.

Debe funcionar con:

- `Integer`
- `String`
- objetos propios

El prompt debe explicar qué significa usar genéricos en Java y por qué la lógica de la pila no depende del tipo de dato almacenado.

Ejercicio 4: Cola genérica

Modificar la implementación de cola para que pueda almacenar cualquier tipo de dato.

Debe probarse con una cola de nombres de personas y con una cola de objetos `Cliente`.

El prompt debe explicar cómo se mantiene el comportamiento FIFO independientemente del tipo de dato.

Ejercicio 5: Historial de navegación con pila

Implementar un sistema simple de historial de navegación usando una pila enlazada.

El programa debe permitir:

- visitar una página;
- volver a la página anterior;
- consultar la página actual;
- imprimir el historial.

El prompt debe explicar por qué una pila representa correctamente el comportamiento de “volver atrás”.

Ejercicio 6: Cola de atención de clientes

Implementar una cola de atención para un banco.

Cada cliente debe tener:

- nombre;
- número de turno;
- motivo de consulta.

El sistema debe permitir:

- agregar cliente a la fila;
- atender al próximo cliente;
- consultar quién sigue;
- imprimir la fila actual.

El prompt debe explicar por qué la estructura adecuada es una cola y cómo se representa el orden de llegada.

Ejercicio 7: Verificador de paréntesis balanceados

Implementar un programa que reciba una expresión y determine si los paréntesis están balanceados.

Ejemplos válidos:

(2 + 3) * (5 - 1)
((a + b) * c)

Ejemplos inválidos:

(2 + 3
())(

El prompt debe explicar cómo se usa una pila para guardar aperturas y validar cierres.

Ejercicio 8: Invertir una palabra con pila

Implementar un programa que invierta una palabra usando una pila enlazada.

Ejemplo:

"algoritmo" → "omtirogla"

El prompt debe explicar por qué la política LIFO permite invertir el orden de los caracteres.

Ejercicio 9: Cola de impresión

Implementar una cola de impresión.

Cada trabajo de impresión debe tener:

- nombre del archivo;
- cantidad de páginas;
- usuario que lo envió.

El sistema debe permitir:

- agregar trabajos;
- imprimir el próximo trabajo;
- consultar el próximo archivo;
- mostrar todos los trabajos pendientes.

El prompt debe explicar por qué una impresora debe procesar los trabajos usando FIFO.

Ejercicio 10: Lista doblemente enlazada simple

Implementar una lista doblemente enlazada genérica.

Debe permitir:

- insertar al inicio;
- insertar al final;
- eliminar un elemento;
- imprimir hacia adelante;
- imprimir hacia atrás.

El prompt debe explicar qué referencias tiene cada nodo, por qué se necesita `anterior` y `siguiente`, y qué casos especiales hay que contemplar al eliminar nodos.